

ENTREGA ANÁLISIS

Integrantes:

Jesus David Casallas - 202514780

Andres Felipe Ramirez Tenjo - 202513379

Contexto del problema

Este proyecto busca construir un sistema que apoye el funcionamiento de una board game cafe, teniendo en cuenta las restricciones del negocio y los requerimientos funcionales del sistema.

Análisis del problema

Para la propuesta de la solución se identificarán las clases que se requieren para el funcionamiento del sistema. También se identificaron las relaciones que hay entre las distintas clases, y por último, se identificaron los posibles atributos de cada clase.

Clases

La primera clase identificada bajo el contexto del problema es la clase **Usuario**, quienes serán los que van a interactuar con el sistema. Esta clase tiene tres subclases que van a ser los diferentes roles que puede tomar cada usuario. La subclase **Administrador** quien es el encargado de las finanzas y asuntos administrativos del board game cafe. La subclase **Cliente** quienes pueden hacer reservas de mesas y juegos dentro del café, y finalmente la clase **Empleado** que a su vez se divide en dos subclases, **Cocinero** y **Mesero**.

Después tenemos la clase **JuegosDeMesa** que son los diferentes juegos disponibles tanto para prestar dentro del café como para vender. Los juegos se dividen en tres categorías, juegos de **Tablero**, juegos de **Acción** y juegos de **Cartas**. Cada juego tiene una dificultad diferente, los juegos con mayor dificultad requieren de un mesero que capacite a los clientes como se debe jugar.

También se tiene la clase **Inventario**, la cual es una clase abstracta de la que heredan las clases **InventarioJuegosDePrestamo**, **InventarioJuegosDeVenta** e **InventarioConsumibles**.

Se tiene la clase **Prestamo** la cual registra los juegos de mesa que fueron prestados en un lapso de tiempo

Se tiene la clase **Producto** de la cual heredan las clases **Bebida**, **Pastelería** y **JuegoDeVenta**.

La clase **Mesa** representa el espacio físico dentro del café. Su función es gestionar la capacidad máxima de la mesa y el estado actual de ocupación.

La clase **Venta** es la encargada de las transacciones de los productos. Además es donde se calcula el impuesto para cada tipo de producto.

La clase **FidelidadDePuntos** gestiona la acumulación y la redención de puntos acumulados por los clientes.

La clase **Turno** representa la jornada laboral de cada empleado.

La clase **Menu** actúa como el catálogo de productos disponibles del café

La clase **Reserva** representa las solicitudes de las mesas hechas por los clientes.

La clase **Sugerencia** permite a los empleados proponer nuevos platos o productos al catálogo.

La clase **SolicitudCambioTurno** permite a los empleados generar una solicitud para cambiar de turno, el administrador es quien aprueba o niega la solicitud bajo la condición de que debe haber 1 cocinero y 2 meseros.

Finalmente la clase **BoardGameCafe** actúa como la clase principal donde coordina la interacción entre el administrador, los usuarios y el inventario. Además contiene la lógica global del funcionamiento del negocio.

Posibles atributos de las clases:

Usuario

id,nombre,email,login,password.

Cliente

fechaNacimiento,puntosFidelidad , códigoDescuento, juegosFavoritos[], historialCompras[].

Empleado

tipo(cocinero,mesero),turno,juegosFavoritos[],juegosDominados[],juegosDominados[](sólo mesero), descuentoEmpleados.

Administrador

nivelAcceso

JuegoDeMesa

id,nombre,añoPublicacion,empresaMatriz,categoria(cartas, tablero,acción),minJugadores,restriccionEdad(menores_5, ningun,adultos), esDifícil:boolean, copiasPrestamo, copiasVenta, precioVenta.

JuegoDeMesa

heredan todo , diferencia por categoría

Mesa

id,capacidad,disponible:boolean, tienenNiños:boolean, tieneJovenes:boolean, numPersonas.

Reserva

Reserva id,fechaHoraInicio, fechaHoraFin, numPersonas tieneNiños, tieneJovenes,estado(activa, finalizada, rechazada).

Prestamo

id, fechaHoraInicio, fechaHoraFin, estado(activo, devuelto, robado),advertenciaSinMesero:boolean.

Producto

id, nombre, precio, descripcion.

Bebida

esAlchol: boolean, esCaliente:boolean.

Pasteleria

alergenos[] (lista de strings)

InventarioJuegosPrestamo

id, juegos[], capacidad

InventarioJuegosVenta

id, juegos[]

InventarioComida

id, productos[]

Venta

id,fecha, subtotal, impuestos, total, puntosGenerados, descuentoAplicado, tipoVenta(Juegos,Comida)

VentaComida

propina ,impuestoConsumo(8%).

VentaJuegos

Iva.

Turno

id, diaSemana (enum), horaInicio, horaFin, empleados[].

SolicitudCambioTurno

id, tipo (GENERAL, INTERCAMBIO), empleadoSolicitante, empleadoDestino (nullable), estado (Pendiente, Aprobado, Rechazado), fechaSolicitud

Sugerencia

id, descripcion, fechaSugerencia, estado (Pendiente, Aprobado, Rechazado)

ProgramaFidelidad

id, puntosAcumulados, equivalenciaPesoPorPunto

BoardGameCafe

nombre, direccion, capacidadMaxima, clientesActuales

Restricciones

Según el contexto del problema y las reglas del negocio sobre las cuales se va a modelar el sistema, se identificaron las siguientes restricciones:

- Se tienen restricciones por edad, puntualmente, juegos aptos para menores de 5 años y juegos exclusivos para adultos.
- No es necesario que un cliente tenga un juego para poder pedir productos.
- El mesero no puede hacer el trabajo del cocinero y viceversa.
- El administrador no puede pedir juegos prestados ni ordenar platillos.
- En la mesa no pueden haber más clientes de lo que su capacidad permite.
- Las bebidas alcohólicas no pueden llevarse a mesas con menores de edad.
- Las bebidas calientes no se pueden llevar a mesas con juegos de mesas de tipo acción.
- El Administrador no puede ordenar ni comprar ningún tipo de producto
- Para que el café funcione debe haber mínimo 1 cocinero y 2 meseros simultáneamente, así que los cambios no se podrán solicitar si hacerlos incumple esta regla.
- Un cliente solo puede reservar hasta dos juegos de mesa simultáneamente.
- El sistema debe aplicar obligatoriamente un 19% de IVA a la venta de juegos y un 8% de Impoconsumo a la comida/bebida.
- Se debe sugerir un 10% de propina en los productos de cafetería, la cual debe calcularse sobre el valor antes de impuestos, aunque el cliente pueda modificarla.
- Los empleados tienen un descuento del 20% y pueden compartir un 10%. Estas dos tasas no pueden sumarse bajo ninguna circunstancia.

Relaciones

Según las clases identificadas y el cómo se relacionan dentro del contexto del problema se identificaron las siguientes relaciones para el funcionamiento del sistema:

- **Préstamo — Mesa (1:1):** Todo préstamo se vincula exclusivamente a una mesa física para garantizar el control de los comensales.
- **Préstamo — JuegoDeMesa (1:1..2):** Un préstamo puede contener un máximo de dos juegos, validando la restricción de capacidad por mesa.
- **Mesa — Reserva (1:N):** Permite el historial de ocupación de cada mesa, asegurando que cada reserva esté asociada a una única ubicación física.
- **Administrador — Sugerencia (1:N):** Centraliza la recepción y aprobación de propuestas de mejora de los empleados.
- **Administrador — SolicitudCambioTurno (1:N):** Gestión administrativa de las peticiones de cambio de horario realizadas por el personal.

- **Empleado — Sugerencia (N:1):** Trazabilidad de las propuestas de nuevos productos por parte del personal.
- **Empleado — SolicitudCambioTurno (1:1):** Relación donde el empleado formaliza su petición de cambio, sujeta a validaciones automáticas.
- **Empleado — Turno (N:1):** Asignación de jornadas que permite al sistema validar la regla operativa (1 cocinero y 2 meseros).
- **BoardGameCafe — Inventario (1:3):** El sistema central encapsula tres instancias independientes de inventarios (Préstamo, Venta, Comida), asegurando la segregación de responsabilidades.
- **Inventario — Producto (1:N):** Define el catálogo de ítems disponibles en cada inventario para su consulta y gestión.
- **Venta — Producto (1:N):** Cada venta se compone de una lista de productos o juegos, permitiendo la aplicación polimórfica de impuestos (IVA 19% o Imptoconsumo 8%) según el tipo de producto.
- **Usuario — ProgramaFidelidad (1:1):** Cada usuario posee un único programa de fidelización asociado para la acumulación del 1% en puntos por cada transacción.

Requerimiento funcional

Requerimiento Funcional de todos los usuarios:

1. todos los usuarios pueden ingresar con su login y contraseña

RF-Cliente:

1. Reservar la mesa con el número de personas e indicando si hay niños o jóvenes.
2. Consultar el catálogo de juego y ver opciones disponibles,
3. Solicitar préstamo de hasta dos juegos y que sean devueltos al final.
4. Consultar menú.
5. Poder comprar juegos del inventario y consultar el historial de compras.
6. Aplicar código de descuento o puntos de fidelidad en una compra
7. gestionar lista de juegos favoritos.

RF-Empleado(cocinero y mesero)

1. Consultar turno semanal y solicitar intercambio.
2. Poder comprar cosas con el código de descuento de empleado.
3. Poder sugerir un nuevo plato.

RF-Mesero

1. Solicitar préstamo de juego cuando está fuera de turno.
2. Poder registrar juego en el que es master.
3. Gestionar lista de juegos favoritos.

RF-Admin

1. Manejar el inventario de préstamo(añadir, reparar, mover a venta, marcar que esta robado)
2. Gestionar inventario de venta (agrega, reabastecer, mover a préstamo)
3. Poder ver todo el historial de préstamo y que se pueda ordenar por: juego, usuario y fecha.
4. Construir modificar turnos semanales y hacer cambios solicitados.
5. Poder agregar y quitar platillos, también aprobar sugerencias de empleados.
6. Consultar informe de ventas y poder filtrar por tipo, por fecha o impuestos y propina.

Cambios segunda entrega

Para esta segunda entrega tomamos algunas decisiones de diseño con respecto al modelo que teníamos para la entrega uno, ya que nos dimos cuenta que teníamos clases que eran innecesarias.

Entre los cambios más importantes decidimos quitar las clases que heredan de JuegoDeMesa, ya que nos dimos cuenta de que nos basamos en según el contexto lo único diferente entre los juegos de mesa son la categoría de juego(tablero, acción y cartas) pero a la hora de revisar el comportamiento de las tres categorías es el mismo por lo cual no había realmente una necesidad de crear tres subclases diferentes. Por eso decidimos que JuegoDeMesa heredara de producto ya que realmente tiene el comportamiento de un producto, y decidimos agregar un atributo enum con las tres categorías de los juegos de mesa.

Otro cambio relevante fue el de tener un único inventario, originalmente teníamos una superclase inventario de la cual heredan tres inventarios diferentes, según si eran juegos para vender, prestar o para productos consumibles. Pero nos dimos cuenta que esto complicaba demasiado el diseño y decidimos unificar los inventarios teniendo una clase única con tres listas, una para los juegos que están a la venta, otra para los juegos que se prestan y por último una lista para los productos consumibles.

También simplificamos el sistema de sugerencias para no tenerlo separado por cambio de turno o incluir platillos nuevos lo que simplifica el diseño y los hace más escalables a largo plazo. Aparte también nos dimos cuenta de que no teníamos un cronograma o jornada donde guardar los turnos de los empleados por lo que implementamos una clase para organizar los turnos de los trabajadores.

Con respecto a la persistencia decidimos implementar un patrón de diseño singleton, lo que nos permite tener una única instancia del game board café la cual va a ser la misma cada vez que se ejecute el programa. Además se implementó una interfaz para convertir los datos en una cadena de texto para poder almacenar los datos en un archivo txt.

Por otro lado, en esta reconstrucción simplificamos las relaciones eliminando conexiones que estaban mal o que eran innecesarias, esto permitió poder organizar mejor las clases según las funcionalidades que se requería que tuvieran realmente,

además, corregimos varias cardinalidades las cuales nos ayudan a poder plantear de una forma más clara nuestro sistema.